PYTHON PROGRAMMING
FINAL EXAMINATION

Date April 11, 2026	**Duration** 12 Hours
Total Marks 100 Marks (5 Sections)	**Recommended Resource** Python Docs (docs.python.org)

Student Name: _____	Roll Number: _____

General Instructions

- Write all code in separate .py files named: section_a.py, section_b.py, etc.
- You may only reference the official Python documentation (docs.python.org). No other online resources.
- Each task specifies marks available. Show all work — partial credit is awarded.
- The case study (Section E) must be submitted as case_study.py with clearly labeled functions/classes.
- Do not use external libraries (pandas, numpy) unless explicitly allowed in the question.
- Comment your code briefly where it helps explain your logic.
- Suggested pacing: Sections A-D = 45 min each, Section E = 60 min, Review = 20 min.

Marks breakdown: Section A = 20 pts | Section B = 20 pts | Section C = 20 pts | Section D = 20 pts | Section E = 20 pts

SECTION A

Python Fundamentals

Data types, variables, operators, string operations — Weeks 1 and 2

20 marks

⏱ ~45 minutes

A1 Variables, Types & Type Conversion — 4 marks — Easy

You are processing student registration data entered by a user. The data comes in as raw strings:

```
raw_name = "  ali raza  "
raw_age = "21"
raw_gpa = "3.75"
raw_enrolled = "True"
```

Write a Python script that:

1. Cleans raw_name (remove extra spaces, convert to title case)
2. Converts raw_age to an integer and raw_gpa to a float
3. Converts raw_enrolled to a proper boolean
4. Prints: Name: Ali Raza | Age: 21 | GPA: 3.75 | Enrolled: True

Expected output: Name: Ali Raza | Age: 21 | GPA: 3.75 | Enrolled: True

A2 String Operations & F-Strings — 5 marks — Easy

Write a program that accepts a sentence from the user and performs these operations, printing each result:

1. Count the total number of words
2. Count how many times the letter 'a' appears (case-insensitive)
3. Reverse the entire string
4. Check if it is a palindrome (ignoring spaces and case)
5. Replace every vowel (a e i o u) with *

Test your program with: "Never odd or even"

Hint: Use .split(), .count(), .replace(), and slicing [::-1]

A3 Arithmetic & Operators — Billing Calculator — 5 marks — Easy

A shopkeeper needs a quick billing calculator. Write a program that takes:

- Item price (float), Quantity (int), Discount % (float), Tax rate (float)

Calculate and print:

1. Subtotal (price x quantity)
2. Discount amount
3. Amount after discount
4. Tax amount (Pakistan GST 17%)
5. Grand total (rounded to 2 decimal places)

Sample: Price=500, Qty=3, Discount=10%, Tax=17% → Grand total: 1579.50

A4 Input Validation — Phone Number

6 marks | **Medium**

Write a program that collects and validates a Pakistani phone number. Rules:

- Must be exactly 11 digits
- Must start with 03
- Must contain only digits (no spaces, dashes, or letters)

The program should:

1. Ask the user to enter a phone number
2. Check all three rules using string methods only (no import re)
3. Print a clear message for each failed rule
4. If valid, print: Valid phone number: 03XX-XXXXXXX (with dash after position 4)

SECTION B

Control Flow & Loops

Conditional logic and loop constructs — Weeks 3 and 4

20 marks

⏱ ~45 minutes

B1 Grade Calculator with Nested Conditions — 5 marks — Easy

Write a program that takes a student's marks (0-100) and assigns a grade and remark:

- 90-100: A+ Outstanding | 80-89: A Excellent | 70-79: B Good
- 60-69: C Average | 50-59: D Pass | Below 50: F Fail

Additionally:

1. If marks are below 0 or above 100, print an error and exit
2. Print the grade and remark
3. If failed, print: "Minimum marks to pass: 50. You need X more marks."

B2 Pattern Printing with Nested Loops — 5 marks — Medium

Using nested loops, print the following patterns. Accept n from input (use n=5 for demo):

Part 1 — Right triangle (2 marks):

```
*
* *
* * *
* * * *
* * * * *
```

Part 2 — Multiplication table grid (3 marks):

```
  1   2   3   4   5
  2   4   6   8  10
  3   6   9  12  15
  4   8  12  16  20
  5  10  15  20  25
```

Hint: Use print(f"{val:4}", end="") for aligned columns.

B3 Number Analysis Loop — 5 marks — Medium

Write a program that asks the user to enter numbers one by one. The user types "done" to stop. After the loop, display:

1. Total count of numbers entered
2. Sum and average (rounded to 2 decimal places)
3. Largest and smallest numbers
4. Count of even vs odd numbers

Handle the case where the user enters a non-numeric value — skip it and print a warning, but don't crash.

B4 FizzBuzz Extended	5 marks	Medium

Print numbers from 1 to 100 applying these rules:

- Divisible by 3 → Fizz
- Divisible by 5 → Buzz
- Divisible by both 3 and 5 → FizzBuzz
- Divisible by 7 → append Boom (or print Boom alone if no other label)
- Otherwise → print the number

After printing all 100 numbers, print a summary showing how many times each label appeared.

SECTION C

Functions

Function design, scope, arguments, advanced function features — Weeks 5 and 6

20 marks

⏱ ~45 minutes

C1 Function Basics & Return Values — 4 marks — Easy

Write the following standalone utility functions, each with a docstring:

- is_prime(n) — returns True if n is prime, False otherwise
- celsius_to_all(c) — returns a tuple of (fahrenheit, kelvin)
- count_vowels(text) — returns the count of vowels in a string
- power(base, exp=2) — returns base raised to exp. Default exponent is 2.

Call each function with at least one test case and print the result.

C2 Using *args and **kwargs — 5 marks — Medium

Write a function generate_report(**student_info) that accepts keyword arguments and prints a formatted report card. It must:

- Accept fields: name, roll_no, marks (a list), subjects (a list)
- Compute the average of marks internally
- Print each field on its own line
- Print the grade based on average (use your B1 grade logic)

Also write total_marks(*args) that accepts any number of integers and returns their sum.

```
generate_report(
    name="Sana Khan",
    roll_no="CS-101",
    subjects=["Math", "Physics", "Python"],
    marks=[85, 78, 92]
)
```

C3 Lambda, map, filter, sorted — 5 marks — Medium

Given this list (do not modify it):

```
students = [
    {"name": "Ali",   "marks": 55, "city": "Karachi"},
    {"name": "Sara",  "marks": 90, "city": "Lahore"},
    {"name": "Umar",  "marks": 45, "city": "Karachi"},
    {"name": "Hina",  "marks": 78, "city": "Islamabad"},
    {"name": "Bilal", "marks": 63, "city": "Lahore"},
]
```

Using lambda, map, filter, and sorted — NO for loops:

- Create a list of just names using map

- Filter only students who passed (marks >= 50) using filter
- Sort students by marks descending using sorted
- Use map to create list of strings like "Ali - 55 marks"
- Use filter to get only students from Karachi

Print the result of each operation.

C4 Recursion | 6 marks | Hard

Part 1 — Fibonacci (2 marks):
Write fibonacci(n) recursively. Print the first 10 Fibonacci numbers.

Part 2 — Factorial (2 marks):
Write factorial(n) recursively. Handle n=0 and negative input gracefully.

Part 3 — Flatten a nested list (2 marks):
Write flatten(lst) that takes a deeply nested list and returns a flat list using recursion.

```
flatten([1, [2, [3, 4], 5], [6, 7]])
# Expected: [1, 2, 3, 4, 5, 6, 7]
```

SECTION D

Data Structures, OOP & File Handling

Lists, dicts, sets, tuples, classes, file I/O — Weeks 7, 8, and 9

20 marks

⏱ ~45 minutes

D1 List & Dictionary Operations — 5 marks — Medium

You have a dictionary of products in a store:

```
inventory = {
    "Laptop":     {"price": 85000, "qty": 10, "category": "Electronics"},
    "T-Shirt":    {"price": 1200,  "qty": 50, "category": "Clothing"},
    "Headphones": {"price": 5500,  "qty": 20, "category": "Electronics"},
    "Jeans":      {"price": 3200,  "qty": 30, "category": "Clothing"},
    "Mouse":      {"price": 1800,  "qty": 15, "category": "Electronics"},
}
```

Write code to:

- Print the total inventory value (price x qty for each, summed)
- Find and print the most expensive item
- List all items in the "Electronics" category
- Apply a 10% discount to all items where price > 5000 (update dictionary)
- Print a sorted list of items by price (ascending)

D2 Sets & Tuples — 4 marks — Easy

Two sections submitted assignments:

```
section_a = {"Ali", "Sara", "Umar", "Hina", "Zara", "Kamran"}
section_b = {"Sara", "Bilal", "Hina", "Imran", "Zara", "Nadia"}
```

Using set operations (no loops), find:

- Students in BOTH sections (intersection)
- Students ONLY in Section A
- All unique students across both sections (union)
- Students in exactly ONE section (symmetric difference)

Also: create a tuple of the three highest-scoring subjects. Then demonstrate tuples are immutable by trying to change a value and catching the error with try/except.

D3 Object-Oriented Programming — Library System — 7 marks — Hard

Part 1 — Book class (3 marks):

Create a Book class with:

- Attributes: title, author, isbn, is_available (default True)
- __str__ method that returns a readable string

- checkout() and return_book() methods that toggle availability

Part 2 — Library class (4 marks):
Create a Library class that:

- Stores a list of Book objects
- add_book(book) — adds a book
- find_by_author(author) — returns all books by an author
- available_books() — returns all currently available books
- total_books() — returns count of all books

Create at least 4 Book objects, add to Library, checkout 2, then display all available books.

D4 File Handling | 4 marks | Medium

Write a script that manages a text-based student record file called students.txt.

- save_student(name, marks) — appends a student record in format: Ali,85
- load_students() — reads file and returns list of dicts: [{"name": "Ali", "marks": 85}, ...]
- top_student() — reads file and returns name of student with highest marks

Save at least 5 students, then call top_student() and print the result.

Use 'with open(...) as f:' for all file operations. Handle FileNotFoundError.

SECTION E

Case Study — Student Management System

Comprehensive project tying together all course concepts — Weeks 1 through 10

20 marks

⏱ ~60 minutes

This is a single integrated task. Build the complete system in one file: case_study.py. Marks are awarded per feature.

E1 Console-Based Student Management System | 20 marks | Hard

You are hired as a junior developer to build a Student Management System for a college. The system must run in the terminal using a menu-driven interface.

Feature 1 — Student Class (4 marks):

Create a Student class with:

- Attributes: roll_no, name, marks (dict of subject:score)
- average() — returns the average of all subject marks
- grade() — returns A/B/C/D/F based on average
- __str__ — returns a neat single-line summary

Feature 2 — Data Storage & File Persistence (4 marks):

Save and load student data from records.txt. Format each line as CSV:

```
CS001,Ali Khan,Math:85,Physics:78,Python:92
```

- save_all(students) — writes all student records to file
- load_all() — reads file and returns list of Student objects
- Data must persist between program runs

Feature 3 — Menu System with Input Validation (4 marks):

Build a loop-driven menu:

```
=== Student Management System ===
1. Add new student
2. View all students
3. Search by roll number
4. Delete a student
5. Show top performer
6. Show class statistics
0. Exit
```

Validate all inputs: non-empty names, numeric marks 0-100, unique roll numbers.

Feature 4 — Analytics (4 marks):

The 'Show class statistics' option must display:

- Total number of students
- Class average (average of all students' averages)
- Highest and lowest scoring students
- Number of students in each grade (A, B, C, D, F)

- The subject with the highest overall average across all students

Feature 5 — Code Quality (4 marks):
Marks awarded for:

- Using functions/methods cleanly — no giant main block
- Proper use of OOP with meaningful methods
- Handling all exceptions gracefully with try/except
- Clear, readable code with at least one docstring per function

Tip: Start by designing the Student class and file I/O functions, then build the menu last. Test each feature before moving on.

— END OF EXAM PAPER — Good luck! —